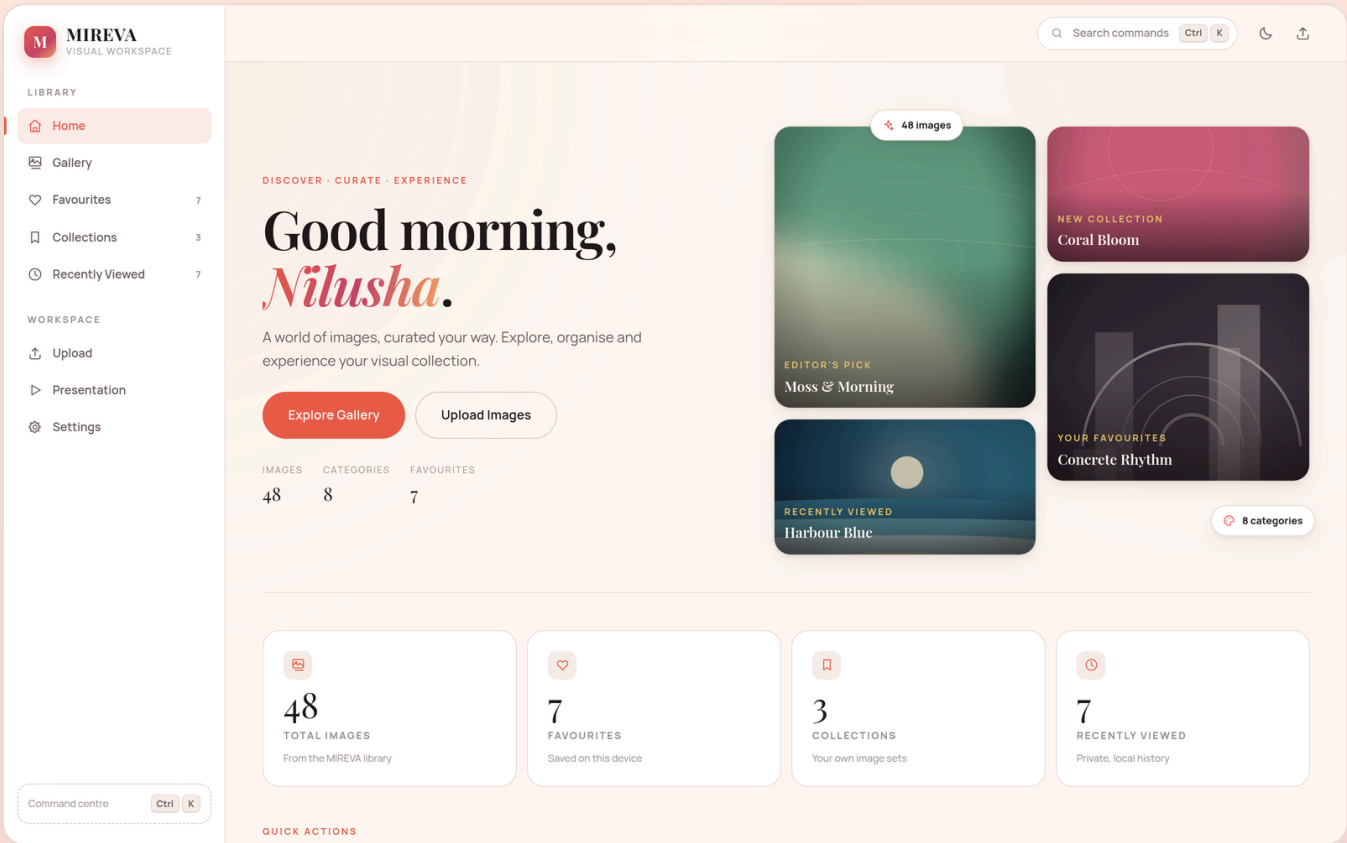


MIREVA

DISCOVER · CURATE · EXPERIENCE

Premium Visual Gallery & Creative Image Workspace

- HTML5
- CSS3
- VANILLA JAVASCRIPT ES6+
- PWA · OFFLINE
- NO FRAMEWORKS



AUTHOR
Nilusha
Madhuwanthi

DOMAIN
Frontend
Development

TASK
Task 1 – Image
Gallery

BUILT WITH
HTML · CSS ·
JavaScript

A real product, not a course exercise.

MIREVA started from a four-line brief — “build an image gallery with next and previous buttons” — and was engineered into a complete visual workspace: a curated library, live search and filtering, a cinematic lightbox, favourites and collections, local uploads, an image editor, presentation mode, and full offline support. Every feature is real and every number below is measured.

8

PAGES

22

JS MODULES

17

STYLESHEETS

48

LIBRARY IMAGES

2

STORAGE LAYERS

0

RUNTIME DEPS

0

CONSOLE ERRORS

165

CACHED FOR OFFLINE

100%

CLIENT SIDE

What the brief asked for — and where it lives

CODEALPHA REQUIREMENT

STATUS

IMPLEMENTATION

Image gallery (HTML/CSS)

Done

Masonry, grid and focus layouts

JavaScript navigation

Done

GalleryManager · LightboxController

Previous / Next buttons

Done

On-screen arrows, arrow keys, swipe

Lightbox view

Done

Full screen, zoom, pan, metadata, filmstrip

Hover effects

Done

Lift, image zoom, caption & favourite reveal

Smooth transitions

Done

200–400 ms system, reduced-motion aware

Responsive design

Done

Phone → tablet → laptop → ultrawide

Image filters / categories

Done

8 categories + orientation, aspect, favourites, recency

How MIREVA differs from an ordinary image gallery.

Most gallery projects are a grid, a modal and two arrows. MIREVA is a workspace: it has state, memory, ownership and an opinion about design.

DIMENSION	TYPICAL GALLERY PROJECT	MIREVA
Content	A handful of hardcoded <code></code> tags	A 48-image data model with categories, tags, descriptions and real metadata — plus your own uploads
Layout	One fixed grid	Masonry, grid and focus, switchable and remembered
Finding things	None, or a category row	Live search across four fields, 8 categories, 6 sort orders, and an advanced filter panel with removable pills
Lightbox	Open, close, next, previous	Zoom, drag-pan, double-click zoom, pinch, swipe, filmstrip, metadata drawer, colour explorer, six keyboard shortcuts
Ownership	Read-only	Favourite, collect, upload, rename, edit, hide, delete — full CRUD
Memory	Resets on refresh	Ten localStorage keys plus an IndexedDB blob store; the app remembers everything
Editing	None	Presets, seven adjustments, rotate, flip, Canvas render, PNG export
Presentation	None	Full-screen slideshow with sources, speeds and progress
Keyboard	Tab order, if lucky	Command centre (⌘K), G-key navigation, full lightbox control, focus trapping
Offline	Breaks	Service worker caches the whole shell — 48 images included
Architecture	One <code>script.js</code>	22 modules with single responsibilities and a shared event bus
Honesty	Fake stats, fake “AI”, invented metadata	Every number is measured locally; missing data says “Metadata unavailable”

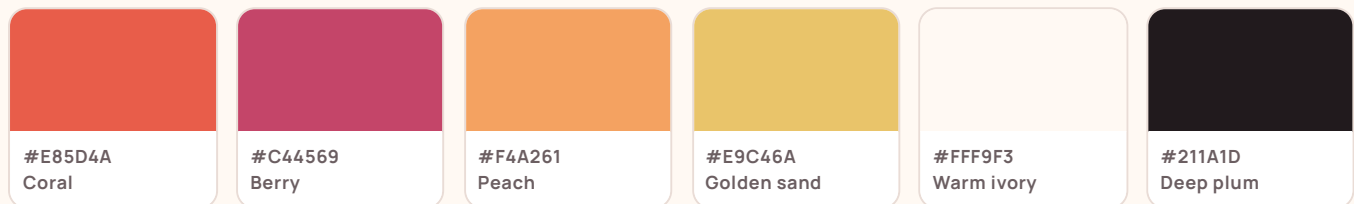
The rule I set for myself: nothing decorative may be fake.

No invented statistics, no “AI-powered” labels on plain filtering, no fake cloud sync badge, no placeholder buttons. View counts are labelled “**Viewed by you**” because MIREVA cannot know global popularity. Where the browser genuinely cannot read a value — a tainted canvas, a missing dimension — the interface says so rather than inventing one.

Warm editorial, not generic SaaS.

MIREVA's visual language is drawn from print photography magazines: a warm ivory paper ground, coral and berry accents, generous negative space, and a display serif set against a precise geometric sans. No neon, no glassmorphism, no dashboard-template blue.

Palette



Dark theme is a separately designed token set — deep plum, charcoal, muted coral and soft peach — not an inversion.

Typography

Playfair Display

Editorial display

Hero headings, page titles, image titles, statistics.
Carries the magazine character.

Manrope

Interface

Navigation, buttons, body copy, metadata, filters.
Quiet, geometric, highly legible at small sizes.

Both are self-hosted as latin-subset `woff2` (204 KB total) so MIREVA makes no third-party font request and works offline.

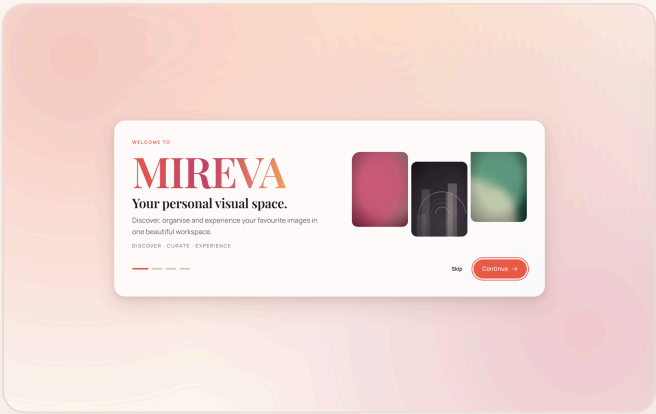
Token discipline

Every colour, space, radius, shadow, duration and easing curve is a CSS custom property declared once in `variables.css`. Theming is a single attribute swap on `<html>`; there is not a single hardcoded hex value scattered through the component styles.

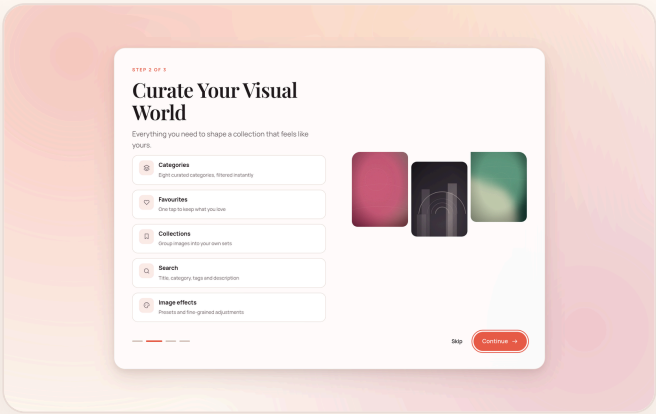
`--primary``--surface-soft``--text-secondary``--sh-2``--r-lg``--dur-3``--ease-out``--tracking-label`

Onboarding & personalisation

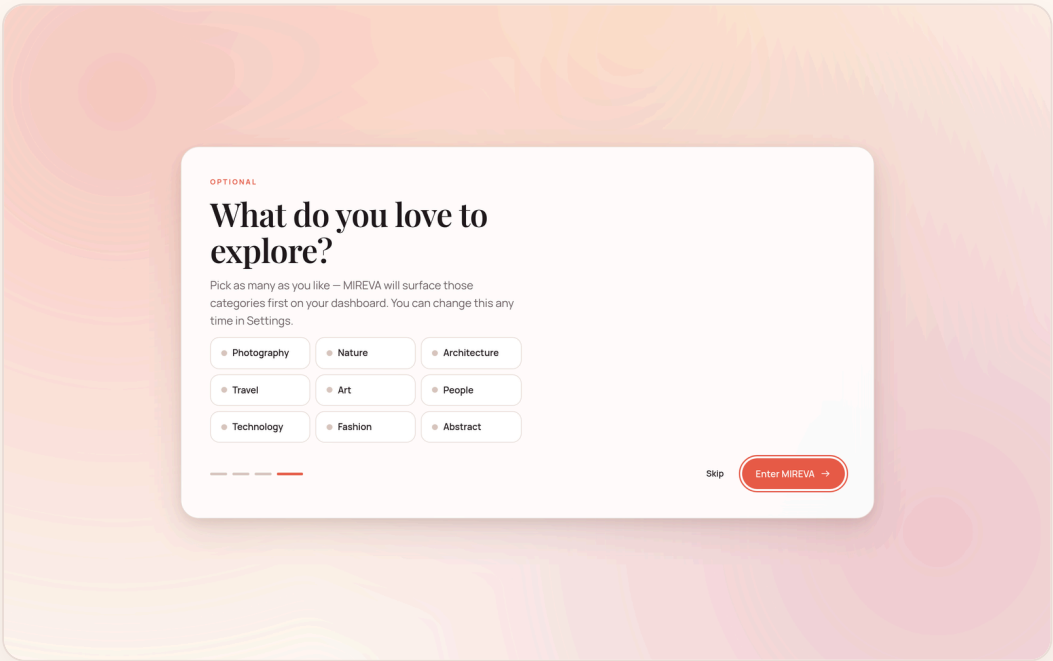
Three editorial screens introduce the product on first visit only. Completion is stored in `localStorage`, progress dots and Skip are always available, and the whole flow can be replayed from Settings. An optional interest picker follows.



Screen 1 — the wordmark, the promise, the positioning.



Screens 2 and 3 — categories, favourites, collections, search, effects, upload, editing.



Personalisation — chosen categories are surfaced first in the dashboard's featured strip. Real stored preferences, described as exactly that; never labelled "AI".

Focus trapped

Tab cannot escape the dialog; focus returns to where it started.

Motion aware

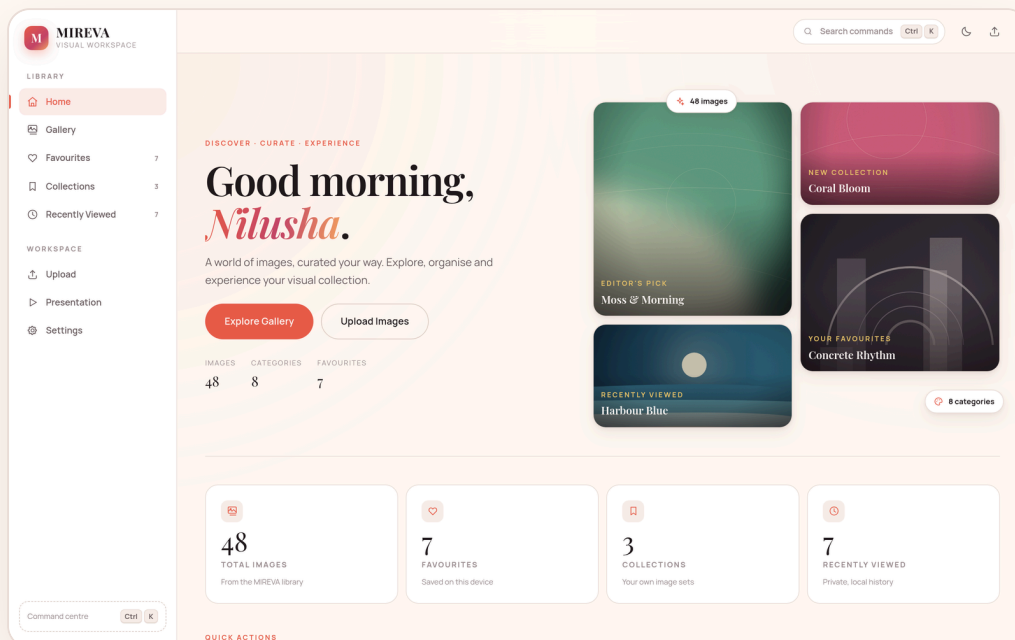
Floating gradients stop entirely under reduced motion.

Shown once

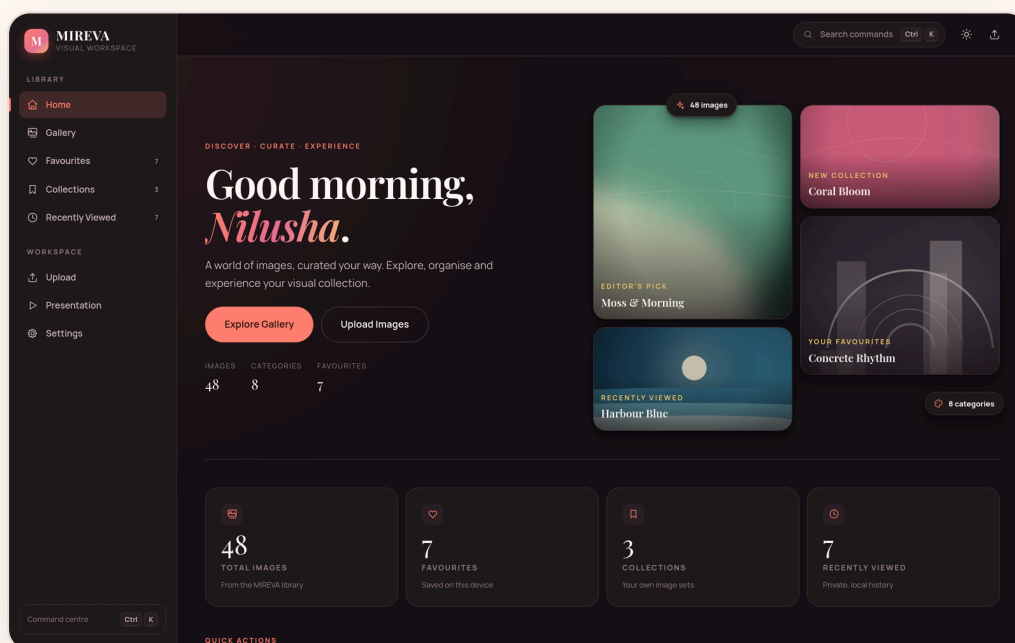
Never nags on a return visit — replayable on demand.

The editorial dashboard

An asymmetric four-image collage with editorial labels, a display-serif greeting that changes with the time of day, real statistics, quick actions with keyboard hints, a featured strip driven by your interests, and your recent history.



Light theme. The collage deliberately picks four different categories so the composition never reads as one colour.



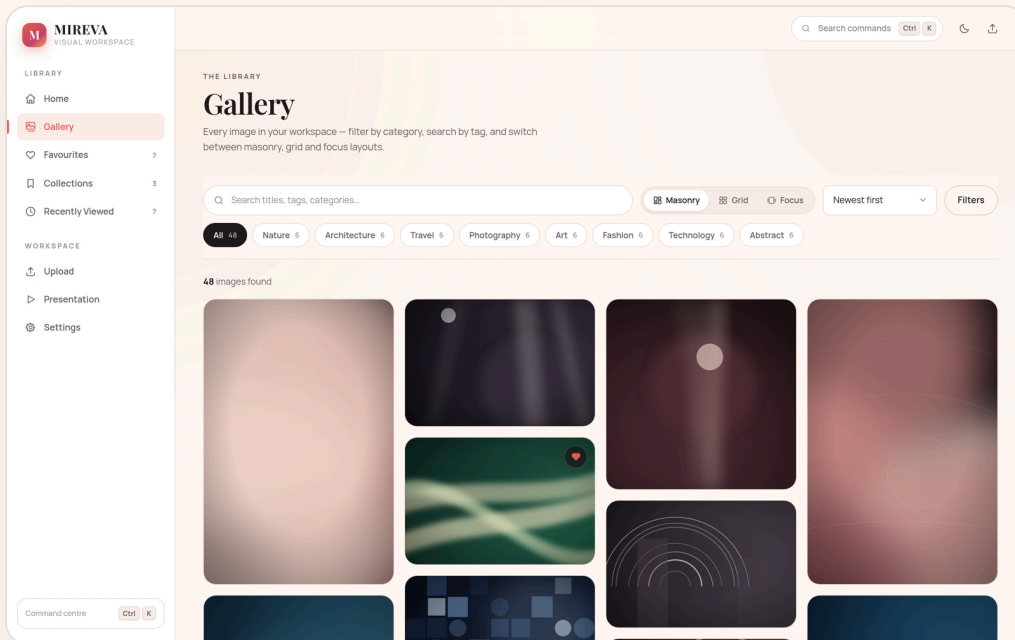
Dark theme — a separately designed palette, not an inverted one. Contrast is maintained across every surface, border and muted text tone.

Every statistic is real

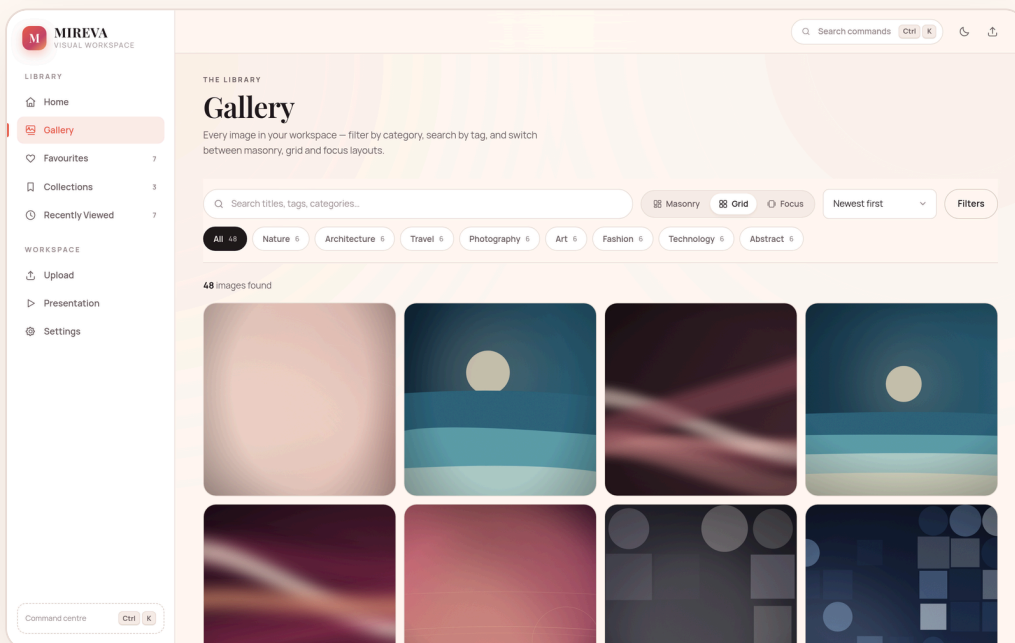
Total images counts the library plus your uploads minus hidden ones. **Favourites**, **collections** and **recently viewed** read straight from storage and are filtered so deleted images never inflate a count. When a value is zero, MIREVA shows a useful empty state instead of a fake number.

Three gallery layouts

Masonry is the primary layout, built with CSS columns so there is no JavaScript measurement loop and no reflow storm. Grid gives a uniform square rhythm. Focus presents one large image at a time with its caption below. The choice is remembered between sessions and can be set as a default in Settings.



Masonry — 2 columns on phones, 3 on tablets and laptops, 4 above 1400 px, 5 on ultrawide.



Grid — uniform squares for fast scanning.

Lazy & CLS-safe

`loading="lazy"`,
`decoding="async"`, intrinsic
 dimensions and `aspect-ratio`
 on every tile.

Delegated events

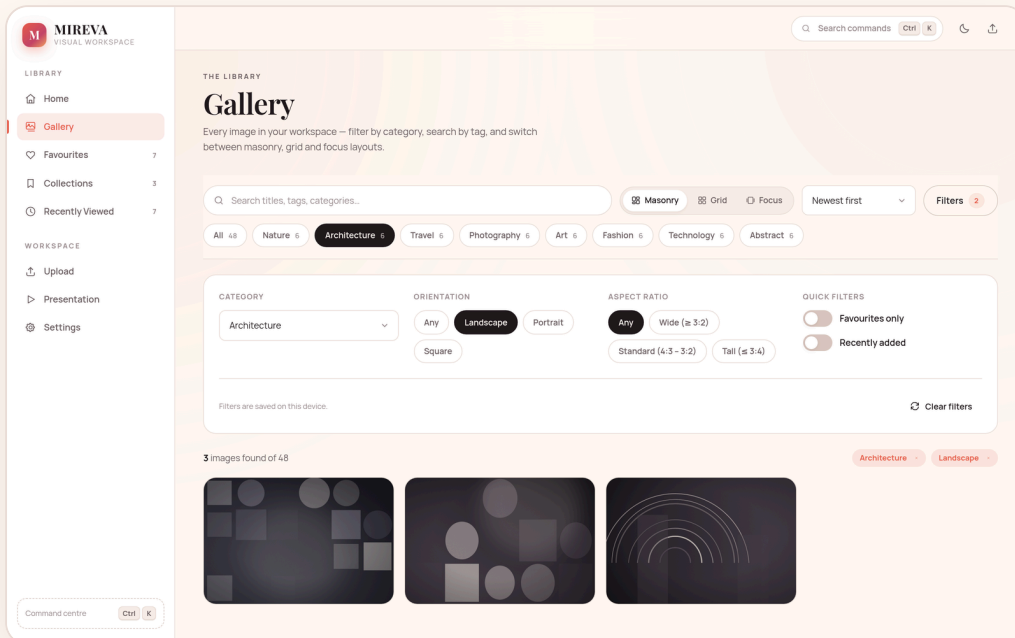
One click and one keydown
 listener per gallery — not one per
 tile.

Skeletons

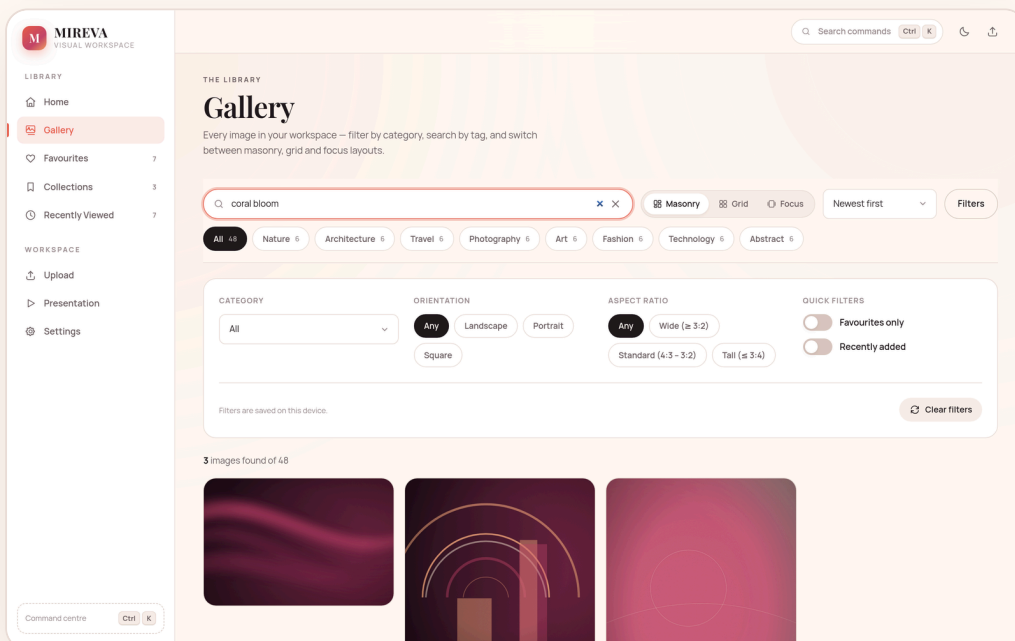
Shimmering placeholders while
 data settles; no spinner soup.

Search, categories, sorting, filters

Search matches every term against **title**, **category**, **tags** and **description** and is debounced so typing never stutters. Categories carry live counts. Six sort orders, an advanced filter panel, and active filters shown as removable pills. The result count is announced to screen readers, and nothing ever reloads the page.



Advanced filters — category, orientation, aspect ratio, favourites-only and recently-added, with a single Clear action.



Live search. When nothing matches, MIREVA says “No visual matches found.” and offers to clear the search and filters in one click.

Sort orders

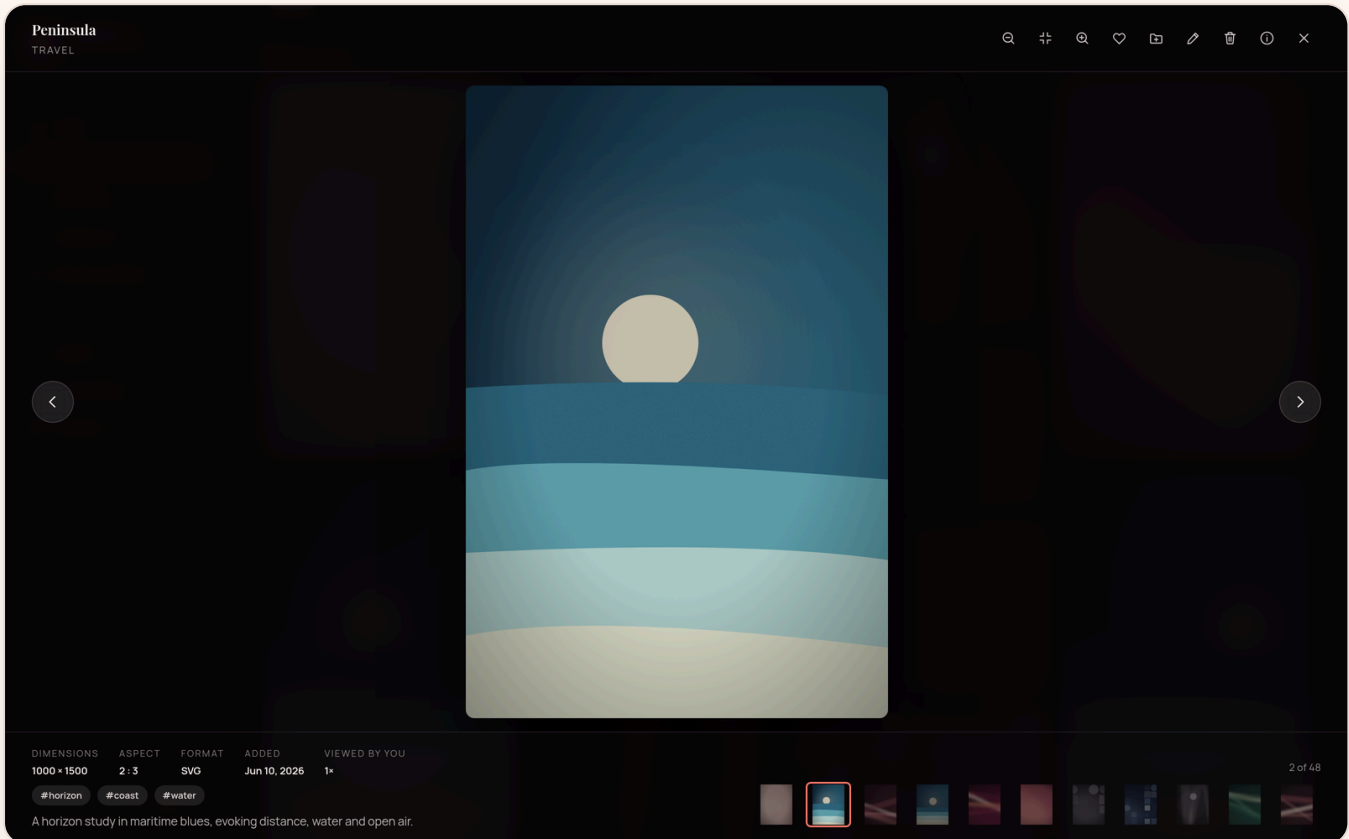
Newest · Oldest · Title A-Z · Title Z-A · Most viewed by you · Favourites first

Persisted

Query, category, orientation, aspect, quick filters and sort are all saved to `mireva_filters` — reopen the gallery and it is exactly as you left it.

The cinematic lightbox

The centrepiece of the CodeAlpha brief, taken as far as it will go: full-screen viewing, previous and next, five zoom levels with drag-to-pan, double-click and pinch zoom, swipe navigation, a filmstrip, favourite, edit, collect and remove — all reachable from the keyboard.



Dark, quiet chrome so the image dominates. Metadata sits along the bottom; the filmstrip tracks your position.

Keyboard map

Esc close

← previous

→ next

+ zoom in

- zoom out

0 reset zoom

F favourite

I info

Touch

Swipe

Left for next, right for previous — with a direction and velocity guard so vertical scrolling is never hijacked.

Pinch

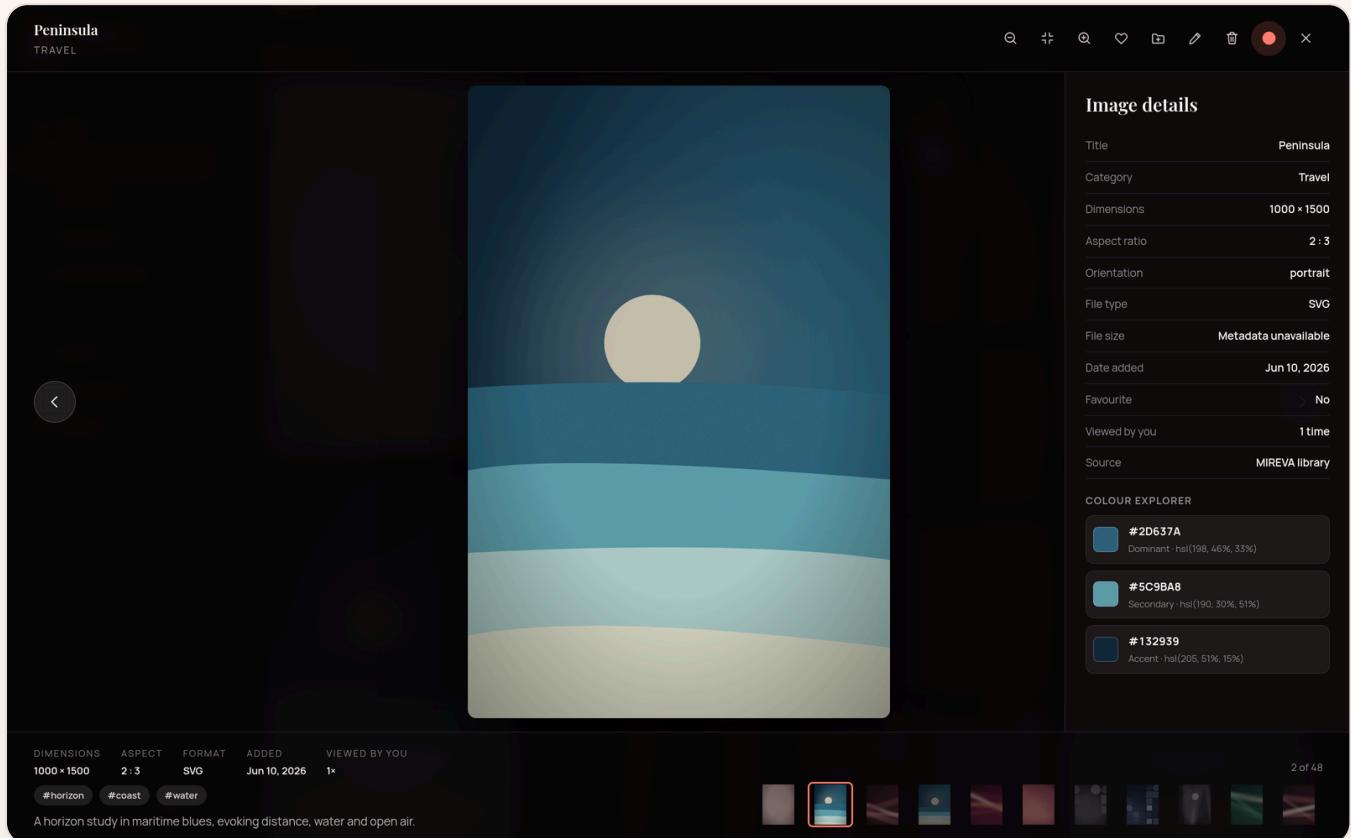
Two-finger pinch maps directly to the zoom scale.

Pan

Pointer Events drag the image once zoomed, with pointer capture for reliability.

Image details & the Colour Explorer

Eleven fields of genuine metadata — title, category, dimensions, aspect ratio, orientation, file type, file size, date added, favourite state, your personal view count and whether the image came from the library or your own uploads.



The details drawer slides in over the stage; the Colour Explorer sits beneath it.

How the colours are found

The image is drawn into a 64 × 64 canvas, every pixel is bucketed into a 4-bit-per-channel histogram, the buckets are ranked by population and then filtered for hue and lightness distance so the three swatches are visually distinct. Click one to copy its hex.

What happens when it can't

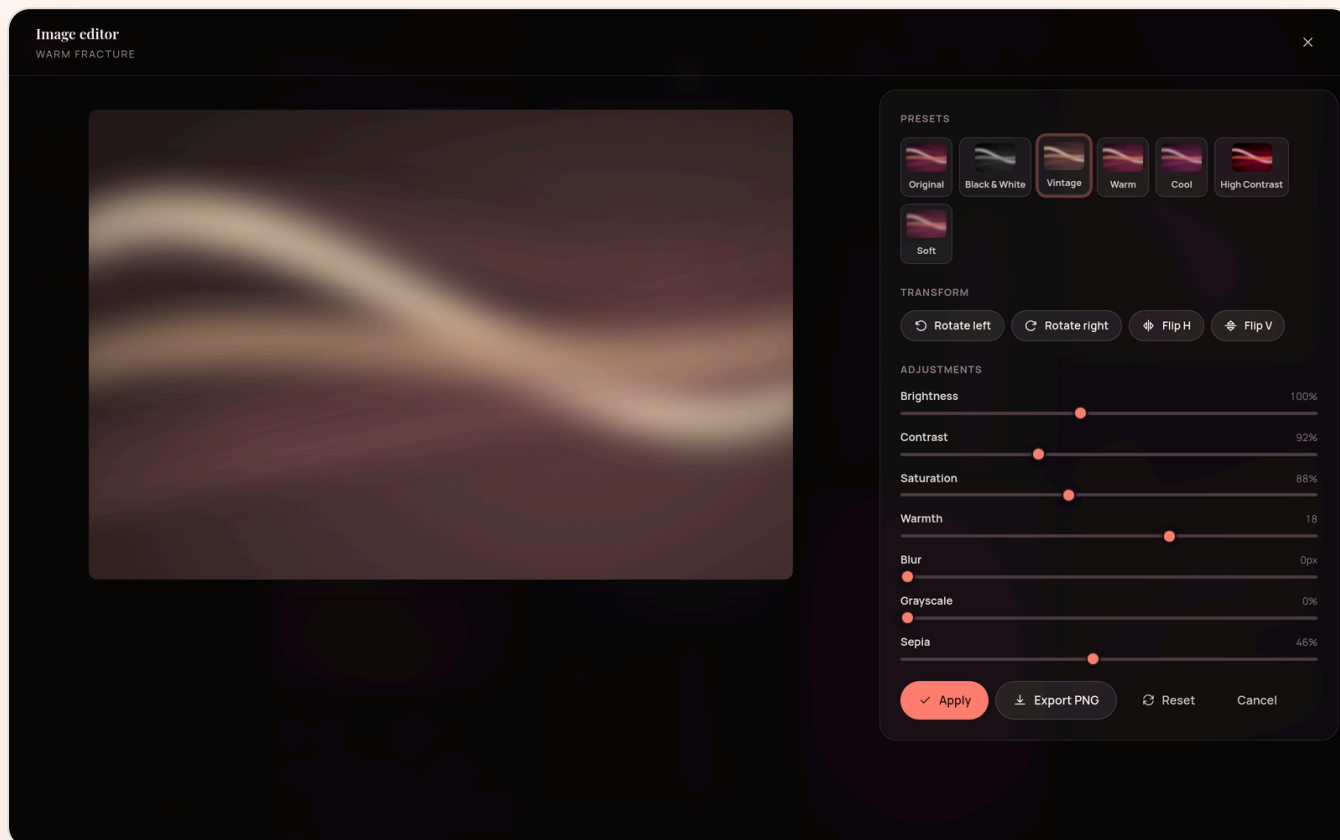
If the canvas cannot be read — a cross-origin image, or the app opened from a `file://` path — MIREVA says **"Colour extraction is not available for this image in your browser."** It never guesses a palette to fill the space.

Honest metadata, in code

Uploaded images without a title are described as **"Untitled Image"** — not given an invented caption. Any field the browser cannot supply renders as **"Metadata unavailable."** This is a deliberate design rule, and it is applied everywhere metadata appears.

Image effects & the editor

Six presets — Original, Black & White, Vintage, Warm, Cool, High Contrast and Soft — plus seven live adjustments and four transforms. The preview uses CSS filters so dragging a slider stays at 60 fps; Apply and Export re-render the result through Canvas at full resolution.



Each preset thumbnail previews its own filter, so you can see the effect before committing.

Adjust

Brightness · Contrast · Saturation
· Warmth · Blur · Grayscale · Sepia

Transform

Rotate left · Rotate right · Flip
horizontal · Flip vertical

Output

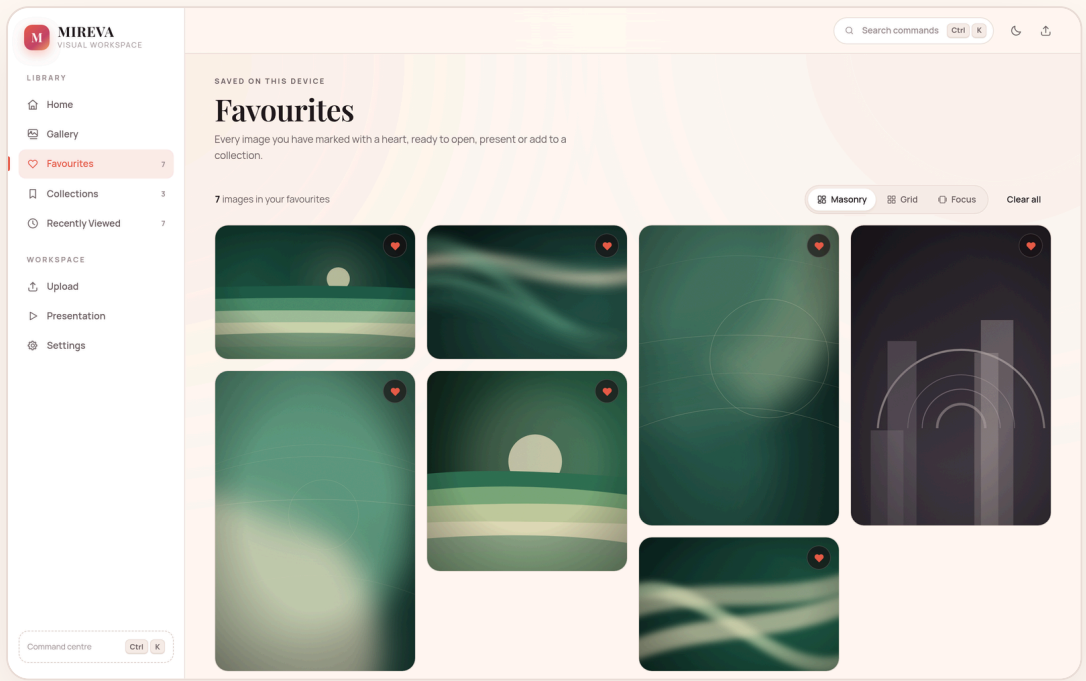
Apply to library · Export PNG ·
Reset · Cancel

Originals are never destroyed

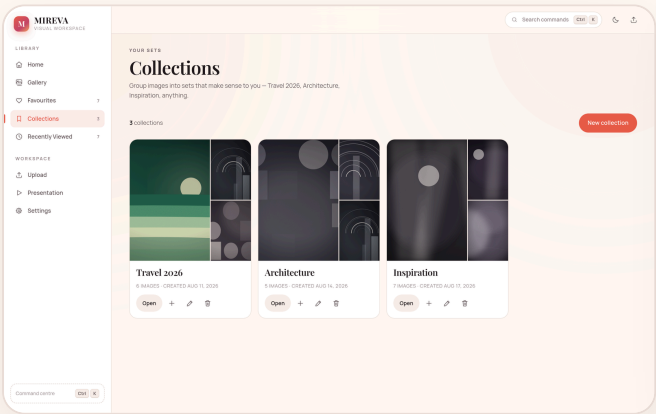
Applying an edit to a library image saves a new copy titled "... (edited)". Applying to one of your own uploads asks whether to replace the original or keep both. Export writes a full-resolution PNG straight from the canvas without touching what is stored.

Favourites, collections & history

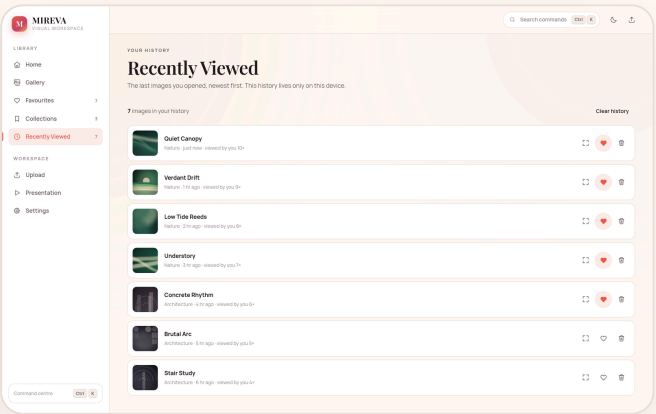
MIREVA is built around ownership. Anything you can see, you can keep, group, revisit, rename or remove — and it is all still there after a refresh, a restart, or a week away.



Favourites — the heart is on every tile, in the lightbox and in every list, with a spring animation and a live sidebar counter.



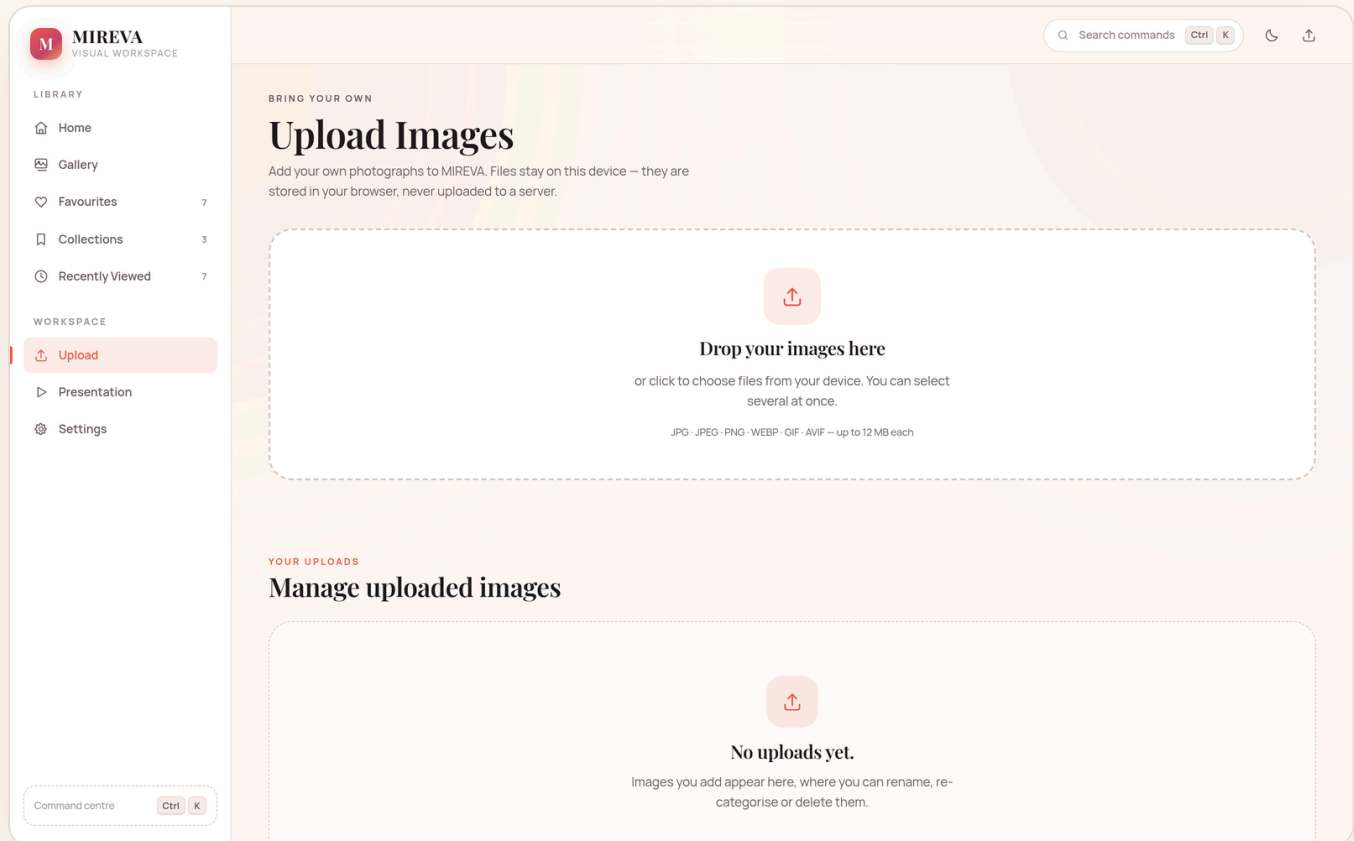
Collections — create, rename, delete, add and remove. Each cover is built live from its own first three images.



Recently viewed — a private local history with relative timestamps and per-image view counts.

Bring your own images

A file picker and a drag-and-drop zone, multiple selection, a per-file title and category, live previews, and validation with human-readable errors. Accepted images are stored as blobs in **IndexedDB** and appear everywhere in the app alongside the library.



The dropzone is also a keyboard-operable button, and a stray drop anywhere else on the page can never navigate the app away.

Validation

- Accepts JPG · JPEG · PNG · WEBP · GIF · AVIF
- Rejects anything over 12 MB, empty files, and files that decode as an image but cannot be read
- Reports each failure per file — the rest of the batch still goes through

Full CRUD afterwards

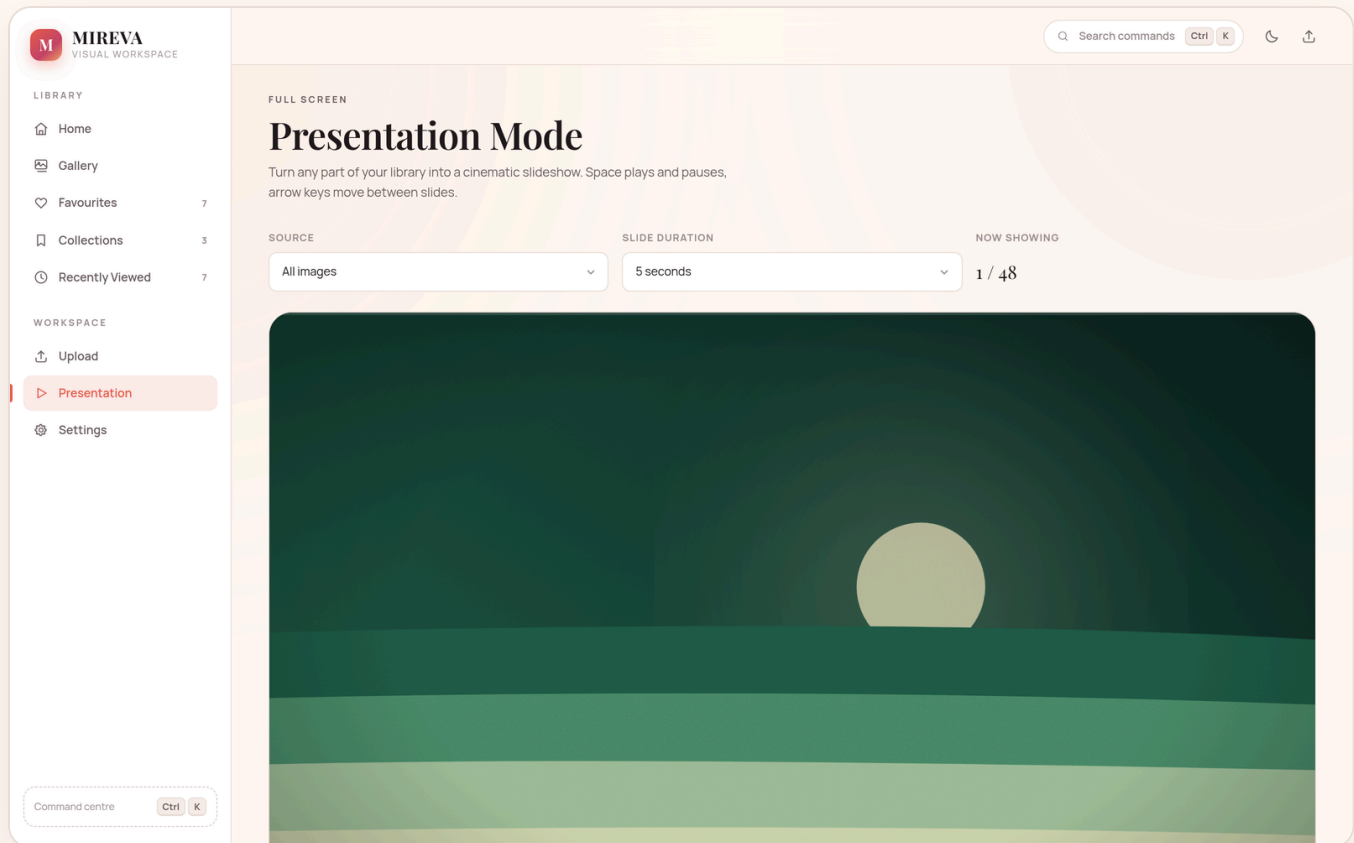
- Rename or re-categorise from the manage list
- Edit through the editor and replace or duplicate
- Delete permanently — favourites, collections and history are cleaned up automatically
- Library images can be hidden and restored from Settings

Nothing leaves the device

There is no server, no upload endpoint and no account. “Upload” here means “into your own browser”, and the interface says so on the page itself.

Presentation mode

Any part of the library can become a full-screen slideshow: all images, a single category, your favourites, your recent history, or any collection you have built.



A blurred, scaled copy of the current image fills the letterboxing so every aspect ratio looks intentional.

Controls

Play, pause, previous, next, 3 / 5 / 8-second speeds and full screen — with a progress bar tied to the real timer.

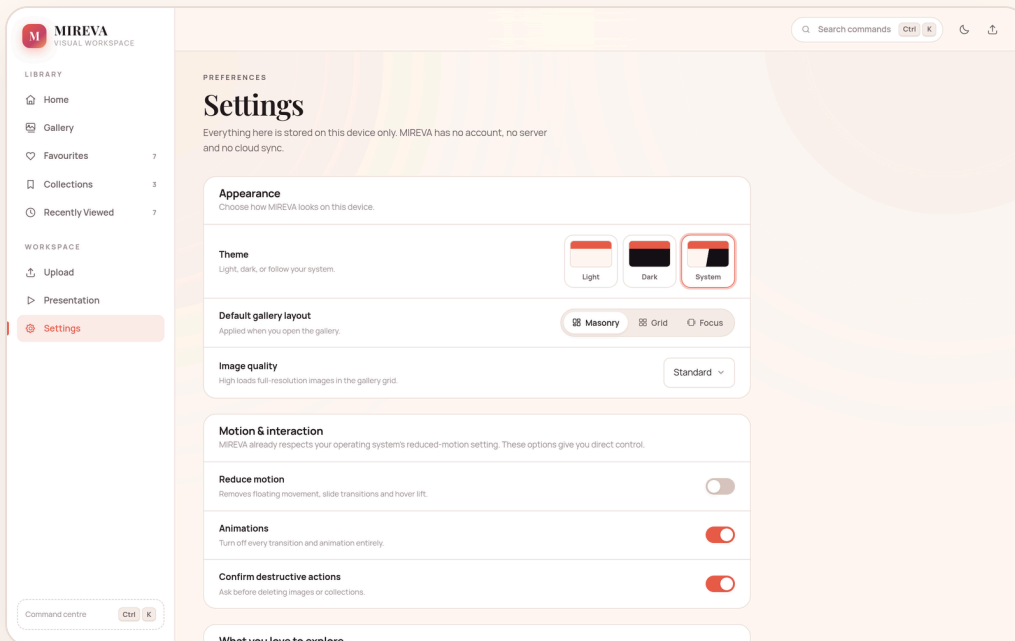
Keyboard

Space toggles playback, arrows step, **Esc** leaves full screen.

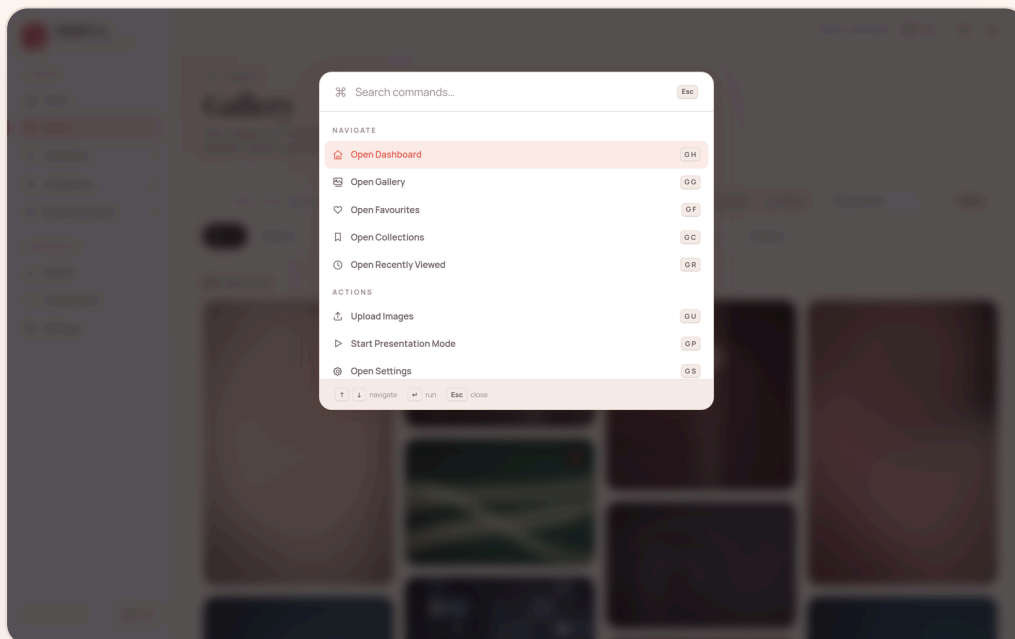
Considerate

The control bar fades out after a few idle seconds while playing, and the Ken Burns drift is removed entirely under reduced motion.

Settings and the command centre

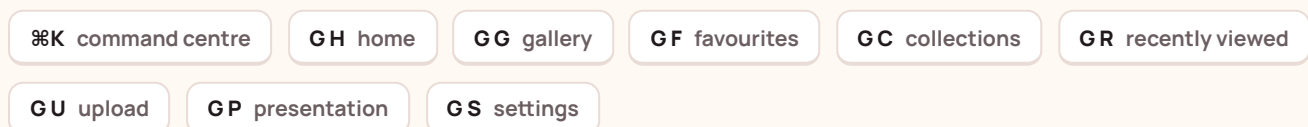


Theme, default layout, image quality, motion switches, destructive confirmations, interests, a live storage meter across both layers, restore hidden images, replay onboarding, and two separated reset actions.



Ctrl / ⌘ + K opens the command centre: filter as you type, move with the arrow keys, run with Enter, dismiss with Escape.

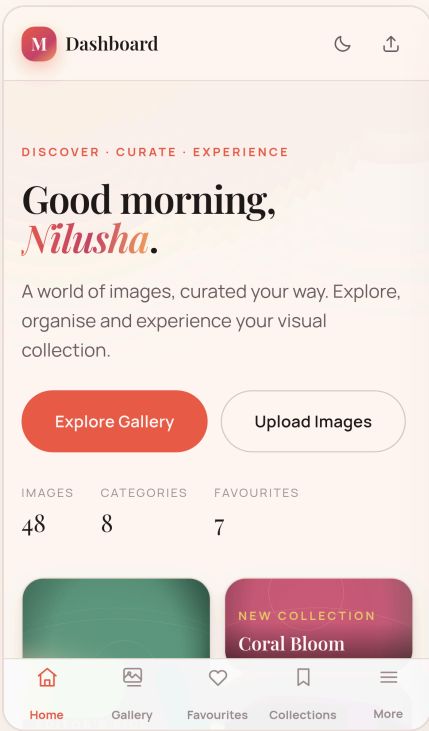
Navigation shortcuts



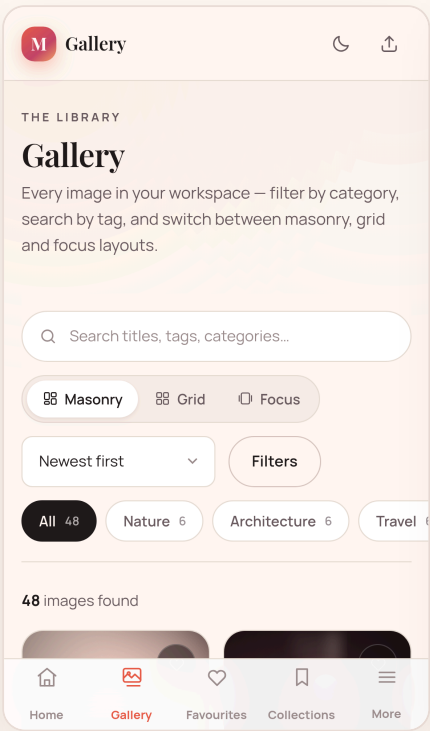
Shortcuts are suppressed while typing in a field and while the lightbox has focus, so they never fight the user.

Designed per breakpoint, not merely shrunk

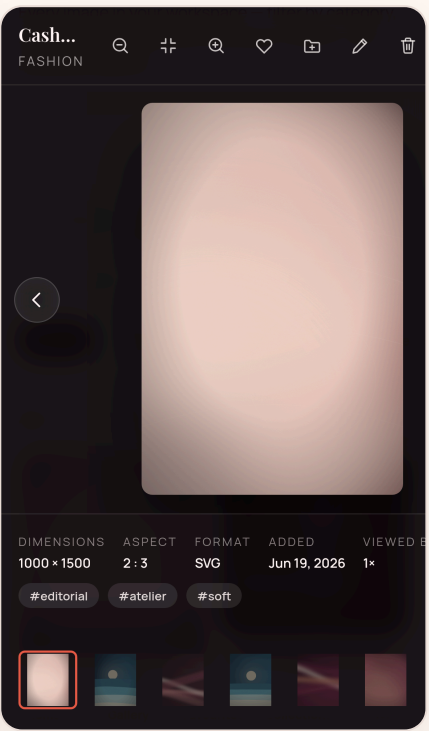
Desktop gets a fixed editorial sidebar and a command trigger in the top bar. Mobile gets a bottom navigation bar with a sheet drawer for secondary pages, always-visible captions and favourite buttons in place of hover states, and larger touch targets.



Dashboard



Gallery



Lightbox

BREAKPOINT

WHAT ACTUALLY CHANGES

≤ 380 px

Tighter type scale and spacing, full-width hero buttons, two-up statistics, portrait collage

≤ 639 px

Bottom navigation, drawer for secondary pages, single-column lightbox footer, floating chips hidden

640 – 1039 px

Three masonry columns, stacked hero, two-up quick actions

≥ 1040 px

Editorial sidebar, two-column hero with collage, command trigger, three-up quick actions

≥ 1400 / 1900 px

Four then five masonry columns

≥ 2100 px

Wider content ceiling and a larger display size for ultrawide monitors

Landscape ≤ 480 px tall

Lightbox footer removed, compact bar, reduced hero padding

Touch devices

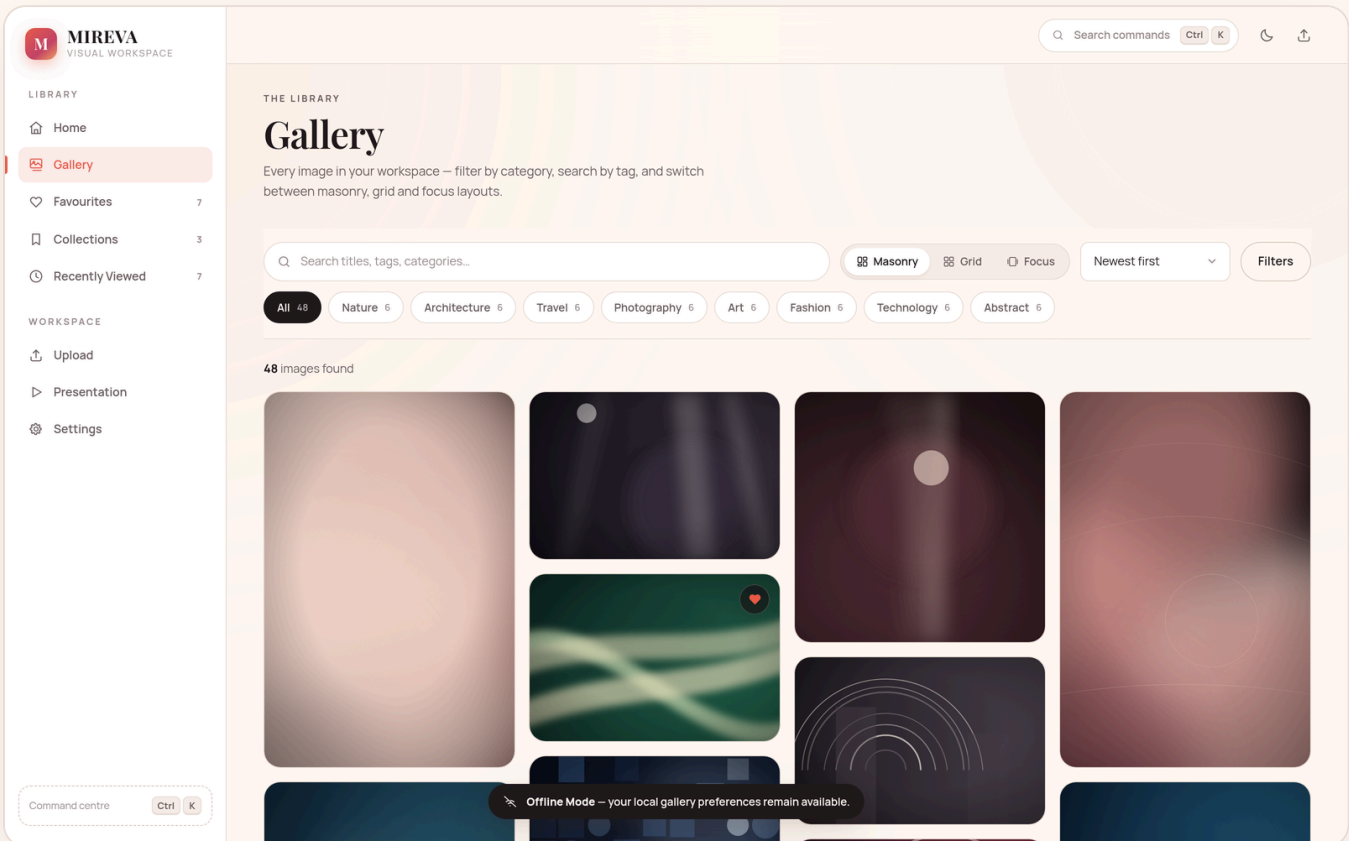
Hover-only affordances resolve to permanent states; no lift on tap

Print

Chrome hidden, tiles kept whole across page breaks

Installable, and genuinely offline

A web app manifest with icons, shortcuts and theme colours makes MIREVA installable. A service worker pre-caches the entire application shell on install — **165 entries**: every page, stylesheet, module, font, icon and library image.



Verified with the network disconnected: all 48 images render, navigation works, and the Offline Mode banner appears.

Caching strategy

- Navigations use network-first with a cache fallback, so a new deploy is picked up quickly
- Assets are cache-first — the library artwork never changes
- Old cache versions are deleted on activate
- Each pre-cache entry is added individually, so one failure cannot break the install

What it does not claim

The offline banner reads “**Offline Mode — your local gallery preferences remain available.**” There is no cloud-sync badge, because there is no cloud. Everything cached is served from this device, and everything you have saved lives on it too.

Architecture

Twenty-two ES modules, each with one responsibility, plus one thin controller per page. No module reaches into another module's internals, and no UI code touches storage directly.

Data

library.js	generated seed data (48 images, 8 categories)
storage.js	StorageService (localStorage) · ImageStore (IndexedDB)
state.js	ImageRepository + AppState + event bus ← single source of truth

Domain

favorites.js	FavoritesManager	collections.js	CollectionManager
filters.js	pure filter + sort	search.js	pure matching
upload.js	UploadController	editor.js	EditorController
presentation.js	PresentationController		

UI

app.js	bootApp() — builds sidebar, top bar, bottom nav, drawer		
gallery.js	GalleryManager (masonry / grid / focus)		
lightbox.js	LightboxController	command-center.js	CommandCenter
onboarding.js	OnboardingFlow	theme.js	ThemeManager
dialogs.js	confirm / prompt / picker / choose		
notifications.js	Toast	accessibility.js	focus trap + announcer
icons.js	inlined 24×24 icon set	utils.js	shared helpers

Pages

js/pages/ dashboard · gallery-page · favorites-page · collections-page
upload-page · presentation-page · recent-page · settings-page

How a single interaction flows

User taps the heart on a tile

→ favorites.js	AppState.toggleFavorite(id)
→ state.js	writes mireva_favorites, emits "favorites:changed"
→ event bus	notifies every subscribed view at once
→ app.js	sidebar counter updates
→ gallery-page.js	list re-renders if a favourites filter is active
→ dashboard.js	statistics and featured strip refresh
→ notifications.js	toast confirms the change
→ accessibility.js	announces it politely to screen readers

One action, one write, one event — and every open view stays correct without any component knowing about any other.

State, storage and resilience

Two storage layers, one abstraction

LAYER	HOLDS	WHY
localStorage via <code>StorageService</code>	Theme, onboarding, preferences, favourites, history, view counts, filters, layout, collections, hidden images	Small, synchronous, JSON-shaped state that every page needs immediately at boot
IndexedDB via <code>ImageStore</code>	Uploaded image blobs and their metadata	Binary data belongs nowhere near <code>localStorage</code> — it would blow the quota and block the main thread

Nothing is allowed to throw at the user

- **Storage unavailable** — private browsing or disabled cookies falls back to an in-memory map so the session still works, and the user is told once
- **Quota exceeded** — caught, surfaced as “Your changes could not be saved locally”, never as a raw `QuotaExceededError`
- **Broken image** — swapped for a neutral placeholder with the alt text “This image could not be displayed”
- **Unreadable upload** — that one file is flagged inline; the rest of the batch still saves
- **Tainted canvas** — the colour explorer and editor explain the limitation instead of failing silently
- **Missing IndexedDB** — the library still works, uploads are simply unavailable
- **No `alert()`, `confirm()` or `prompt()` anywhere** — every one is a styled, focus-trapped `<dialog>`

Performance decisions

Rendering

CSS-column masonry, lazy loading, intrinsic dimensions, lighter SVG thumbnails in lists and full resolution only in the lightbox and editor.

JavaScript

Debounced search, throttled handlers, delegated listeners, cached object URLs revoked on delete, and one `requestAnimationFrame` loop for the slideshow progress bar.

Accessibility and motion

Accessibility was designed in, not retrofitted. Every interactive surface is reachable and operable from the keyboard, every overlay traps and restores focus, and the interface adapts to the user's system preferences before they touch a setting.

Structure & semantics

- Landmarks: `header`, `nav`, `main`, `aside`, `section`, `figure`
- Skip-to-content link as the first focusable element
- Real `<button>` and `<a>` elements — no clickable `div`s, no inline `onclick`
- Native `<dialog>` with `aria-modal` for every modal

State & announcements

- `aria-pressed` on favourites, chips and toggles
- `aria-current="page"` on the active navigation item
- `aria-expanded` on filter and info toggles
- Polite live regions for result counts, zoom level and favourite changes
- Toasts announced through a `role="status"` region

Motion, three ways

System

`prefers-reduced-motion` is honoured before any JavaScript runs.

In-app

A Reduce motion switch removes floating movement, hover lift and slide transitions.

Absolute

An Animations switch disables every transition and animation in the product.

Both settings are written to `<html>` as data attributes by the inline boot script, so the very first painted frame already respects them — there is no flash of animated content.

Honest alternative text

Library images use their real title and category. Uploaded images without a title are described as “Untitled Image”. MIREVA never invents a description of a picture it cannot see.

Skills demonstrated

Each skill below maps to something you can open and inspect in the repository.

SKILL	WHERE TO SEE IT
Semantic HTML5	Landmarks, <code>figure / figcaption</code> , native <code>dialog</code> , meaningful heading order across eight pages
Advanced CSS3	Custom-property design system, CSS-column masonry, <code>color-mix()</code> , <code>aspect-ratio</code> , container-aware layouts, dual themes
Vanilla JavaScript ES6+	Modules, classes, private fields, optional chaining, destructuring, async/await, Promise-based dialogs
DOM manipulation	A typed <code>el()</code> builder, document fragments for batch inserts, delegated events, zero innerHTML for user data
Modular architecture	22 single-responsibility modules, thin page controllers, no circular dependencies
State management	<code>ImageRepository</code> + <code>AppState</code> + a custom event bus driving every view
LocalStorage	<code>StorageService</code> with namespaced keys, JSON safety, availability probing and a usage meter
IndexedDB	<code>ImageStore</code> with versioned schema, indexes, promise-wrapped transactions and quota handling
Image handling	Blob uploads, object-URL lifecycle, Canvas re-render, PNG export, dominant-colour extraction
Responsive design	Nine designed breakpoints plus touch, landscape, high-contrast and print treatments
UI / UX design	An original editorial identity, empty states, skeletons, toasts and micro-interactions
Accessibility	Focus trapping, live regions, ARIA state, keyboard parity, honest alt text
Keyboard interaction	Command centre, G-key navigation, eight lightbox shortcuts, arrow-key list navigation
Touch interaction	Swipe with direction guards, pinch-to-zoom, pointer-capture panning, bottom sheet drawer
Animation	A documented 200–400 ms motion system with three levels of reduction
Performance	Lazy loading, CLS prevention, debouncing, delegation, thumbnail tiers
Error handling	Every failure path caught and translated into human language
PWA	Manifest, service worker, versioned caches, verified offline operation
Tooling & QA	A Python asset generator and an automated headless-browser QA pass

Tested, not assumed

Before release, MIREVA was driven end-to-end in a headless Chromium session across all eight pages, both themes, and desktop and mobile viewports — with the console, the network and the layout under observation the whole time.

O
CONSOLE ERRORS

O
PAGE ERRORS

O
FAILED REQUESTS

O
DEAD BUTTONS

O
BROKEN IMAGES

O
HORIZONTAL SCROLL

Exercised end-to-end

AREA	VERIFIED
Onboarding	All three screens, Skip, progress dots, interest selection, persistence, replay
Gallery	Masonry / grid / focus switching and persistence; skeletons; lazy loading
Search	Matching across four fields, live counts, clear, and the no-results state
Filters	Category, orientation, aspect, favourites-only, recently-added, pills, clear-all
Sorting	All six orders
Lightbox	Open, close, next, previous, zoom in/out/reset, pan, favourite, info, filmstrip, all keys
Colour explorer	Three distinct swatches extracted and copied to the clipboard
Favourites	Add, remove, persistence across reloads, counter propagation
Collections	Create, rename, delete, add images, remove images, live covers
Upload	Picker, drag & drop, validation, previews, removal, IndexedDB write, manage list
Editor	Presets, sliders, rotate, flip, apply, export, reset, cancel
Presentation	Play, pause, next, previous, speeds, sources, progress, full screen
Settings	Themes, layout, quality, motion switches, storage meter, resets
Command centre	Open, filter, arrow navigation, run, close
Responsive	iPhone, tablet, laptop, ultrawide, landscape
PWA	Registration, 165 pre-cached entries, full offline reload with all 48 images

How to open MIREVA

MIREVA is a static site with no build step — but it must be served over `http://` rather than opened as a `file://` path, because service workers, ES modules and Canvas pixel reads all require an origin.

VS Code — recommended

- Open the `MIREVA` folder
- Install the **Live Server** extension (already recommended by the project)
- Right-click `index.html` → **Open with Live Server**

Any terminal

```
python3 -m http.server 5500
# or
npx serve .
```

Then open `http://localhost:5500`.

Suggested demo order

Onboarding → dashboard → gallery → masonry to grid → search → category filter → lightbox → next / previous → zoom → favourite → favourites → create a collection → drag-and-drop upload → image effects → editor → presentation mode → dark mode → command centre → mobile layout → refresh to show persistence → back to the dashboard.

THANK YOU FOR READING

MIREVA

DISCOVER · CURATE · EXPERIENCE

Nilusha Madhuwanthi

CodeAlpha Frontend Development Internship · Task 1 — Image Gallery

Built with HTML5, CSS3 and vanilla JavaScript. No frameworks. No backend. No tracking.